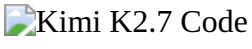
kimi-k2-7-code-hf-model-card

---

tags:

- compressed-tensors license: other license_name: modified-mit library_name: transformers pipeline_tag: image-text-to-text

---



Kimi-Code Homepage Moonshot AI
Hugging Face Moonshot AI Twitter Kimi.ai Discord Kimi.ai ModelScope Moonshot AI
License Modified MIT

1. Model Introduction

Kimi K2.7 Code is a coding-focused agentic model built upon Kimi K2.6. With substantial improvements on real-world long-horizon coding tasks, it strengthens end-to-end task completion across complex software engineering workflows while improving token efficiency, reducing thinking-token usage by approximately 30% compared with Kimi K2.6.

2. Model Summary

Architecture	Mixture-of-Experts (MoE)
Total Parameters	1T
Activated Parameters	32B
Number of Layers (Dense layer included)	61
Number of Dense Layers	1
Attention Hidden Dimension	7168
MoE Hidden Dimension (per Expert)	2048
Number of Attention Heads	64
Number of Experts	384
Selected Experts per Token	8
Number of Shared Experts	1
Vocabulary Size	160K
Context Length	256K
Attention Mechanism	MLA
Activation Function	SwiGLU
Vision Encoder	MoonViT
Parameters of Vision Encoder	400M

3. Evaluation Results

Benchmark	Kimi K2.6	Kimi K2.7 Code	GPT-5.5	Claude Opus 4.8
Coding				
Kimi Code Bench v2	50.9	62.0	69.0	67.4
Program Bench	48.3	53.6	69.1	63.8
MLS Bench Lite	26.7	35.1	35.5	42.8
Agentic				
Kimi Claw 24/7 Bench	42.9	46.9	52.8	50.4
MCP Atlas	69.4	76.0	79.4	81.3
MCP Mark Verified	72.8	81.1	92.9	76.4

▶ **Footnotes**

4. Native INT4 Quantization

Kimi-K2.7-Code adopts the same native int4 quantization method as Kimi-K2-Thinking.

5. Deployment

> [!Note] You can access Kimi-K2.7-Code's API on https://platform.moonshot.ai and we provide OpenAI/Anthropic-compatible API for you. Currently, Kimi-K2.7-Code is recommended to run on the following inference engines:

- vLLM
- SGLang
- KTransformers

Kimi-K2.7-Code has the same architecture as Kimi-K2.5/Kimi-K2.6, and the deployment method can be directly reused.

The version requirement for `transformers` is `>=4.57.1, <5.0.0`.

Deployment examples can be found in the Model Deployment Guide.

---

6. Model Usage

The usage demos below demonstrate how to call our official API. Note that Kimi-K2.7-Code forces thinking and preserve_thinking as True.

For third-party APIs deployed with vLLM or SGLang, please note that:

> [!Note]
>
> - Chat with video content is an experimental feature and is only supported in our official API for now.
> - The recommended `temperature` will be `1.0` for Thinking mode.
> - The recommended `top_p` is `0.95`.
> - Instant mode is not supported.

Chat Completion

This is a simple chat completion script which shows how to call K2.7-Code API in Thinking mode.

```
import openai
import base64
import requests
def simple_chat(client: openai.OpenAI, model_name: str):
    messages = [
        {'role': 'system', 'content': 'You are Kimi, an AI assistant created by Moonshot
        AI.'},
        {
            'role': 'user',
            'content': [
                {'type': 'text', 'text': 'which one is bigger, 9.11 or 9.9? think
        carefully.'}
            ],
        },
    ]
    response = client.chat.completions.create(
        model=model_name, messages=messages, stream=False, max_tokens=4096
    )
    print('====== Below is reasoning content in Thinking Mode ======')
    print(f'reasoning content: {response.choices[0].message.reasoning}')
    print('====== Below is response in Thinking Mode ======')
    print(f'response: {response.choices[0].message.content}')
```

Chat Completion with visual content

K2.7-Code supports Image and Video input.

The following example demonstrates how to call K2.7-Code API with image input:

```
import openai
import base64
import requests

def chat_with_image(client: openai.OpenAI, model_name: str):
    url = 'https://huggingface.co/moonshotai/Kimi-K2.7-Code/resolve/main/figures/kimi-
        logo.png'
    image_base64 = base64.b64encode(requests.get(url).content).decode()
    messages = [
        {
            'role': 'user',
            'content': [
                {'type': 'text', 'text': 'Describe this image in detail.'},
                {
                    'type': 'image_url',
                    'image_url': {'url': f'{image_base64}'},
                },
            ],
        }
    ]

    response = client.chat.completions.create(
        model=model_name, messages=messages, stream=False, max_tokens=8192
    )
    print('====== Below is reasoning content in Thinking Mode ======')
    print(f'reasoning content: {response.choices[0].message.reasoning}')
```

```
    print('====== Below is response in Thinking Mode ======')
    print(f'response: {response.choices[0].message.content}')
```

The following example demonstrates how to call K2.7-Code API with video input:

```
import openai
import base64
import requests

def chat_with_video(client: openai.OpenAI, model_name:str):
    url = 'https://huggingface.co/moonshotai/Kimi-K2.7-
        Code/resolve/main/figures/demo_video.mp4'
    video_base64 = base64.b64encode(requests.get(url).content).decode()
    messages = [
        {
            "role": "user",
            "content": [
                {"type": "text","text": "Describe the video in detail."},
                {
                    "type": "video_url",
                    "video_url": {"url": f"data:video/mp4; base64,{video_base64}"},
                },
            ],
        }
    ]

    response = client.chat.completions.create(model=model_name, messages=messages)
    print('====== Below is reasoning content in Thinking Mode ======')
    print(f'reasoning content: {response.choices[0].message.reasoning}')
    print('====== Below is response in Thinking Mode ======')
    print(f'response: {response.choices[0].message.content}')
```

Preserve Thinking

Kimi K2.7 Code forces `preserve_thinking` mode, which retains full reasoning content across multi-turn interactions and enhances performance in coding agent scenarios.

This feature is enabled by default and can't be disabled. The following example demonstrates how to call K2.7-Code API in `preserve_thinking` mode:

```
def chat_with_preserve_thinking(client: openai.OpenAI, model_name: str):
    messages = [
        {
            "role": "user",
            "content": "Tell me three random numbers."
        },
        {
            "role": "assistant",
            "reasoning_content": "I'll start by listing five numbers: 473, 921, 235, 215,
        222, and I'll tell you the first three.",
            # Some API (e.g. vLLM) may not support reasoning_content, you can try reasoning
        instead
            "content": "473, 921, 235"
        },
        {
            "role": "user",
            "content": "What are the other two numbers you have in mind?"
        }
    ]
```

```
    response = client.chat.completions.create(
        model=model_name,
        messages=messages,
        stream=False,
        max_tokens=4096,
    )
    # the assistant should mention 215 and 222 that appear in the prior reasoning content
    print(f"response: {response.choices[0].message.reasoning}")
    return response.choices[0].message.content
```

Interleaved Thinking and Multi-Step Tool Call

K2.7-Code shares the same design of Interleaved Thinking and Multi-Step Tool Call as K2 Thinking. For usage example, please refer to the K2 Thinking documentation.

Coding Agent Framework

Kimi K2.7-Code works best with Kimi Code CLI as its agent framework — give it a try at https://www.kimi.com/code.

---

7. License

Both the code repository and the model weights are released under the Modified MIT License.

---

8. Third Party Notices

See THIRD PARTY NOTICES

---

9. Contact Us

If you have any questions, please reach out at support@moonshot.ai.